

# **COL334 Assignment 3**

Raw TCP/IPv4 and HTTP over a four-machine network

Report

Team members (entry numbers):  
2020CS10366 and 2020CS10367

Contents: R1 Network setup | R2 Traceroute | R3 Reverse-path filtering | R4 Our client  
with a standard server | R5 Standard clients with our server | R6 Packet capture and TCP  
mechanics

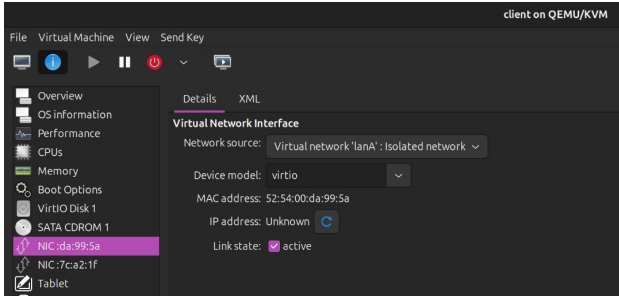
## R1. Network setup

We built the network with four Ubuntu 22.04 virtual machines: **client**, **r1**, **r2** and **server**. We first tried VirtualBox, but it would not build on our host's new Linux kernel, so we used KVM/QEMU with virt-manager instead. The four networks (lanA, lanB, lanC and linkD) are isolated virtual networks that we made with virsh. DHCP and forwarding are switched off on all of them, so the machines can only talk to each other and not to the host. Every machine uses virtio network cards. The table shows which card is attached to which network and which address we gave it.

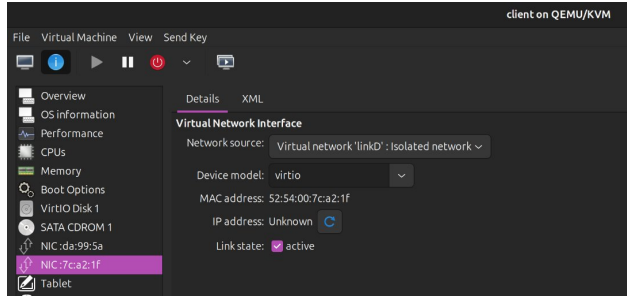
| Machine | Interface | Network | Address       |
|---------|-----------|---------|---------------|
| client  | enp1s0    | lanA    | 10.10.1.10/24 |
| client  | enp7s0    | linkD   | 10.10.4.2/30  |
| r1      | enp1s0    | lanA    | 10.10.1.1/24  |
| r1      | enp7s0    | lanB    | 10.10.2.1/24  |
| r2      | enp1s0    | lanB    | 10.10.2.2/24  |
| r2      | enp7s0    | lanC    | 10.10.3.1/24  |
| r2      | enp8s0    | linkD   | 10.10.4.1/30  |
| server  | enp1s0    | lanC    | 10.10.3.10/24 |

### Network-adaptor settings of the four VMs

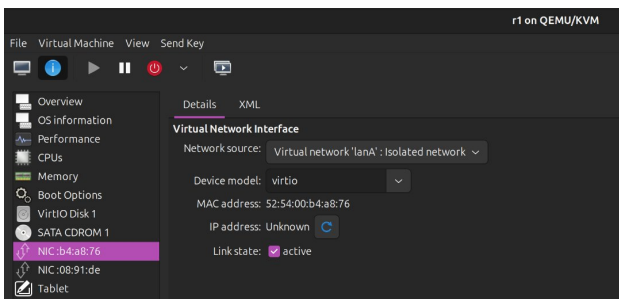
These virt-manager screenshots show the network each card is connected to. Every card is an isolated virtual network, with the virtio model.



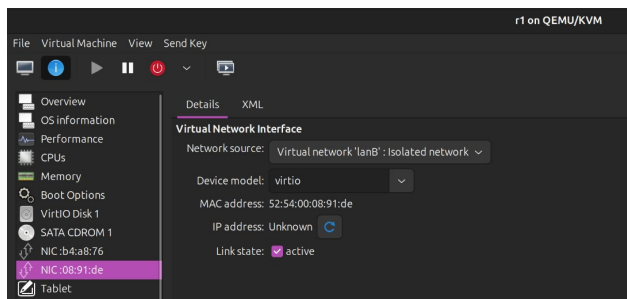
client: card on lanA



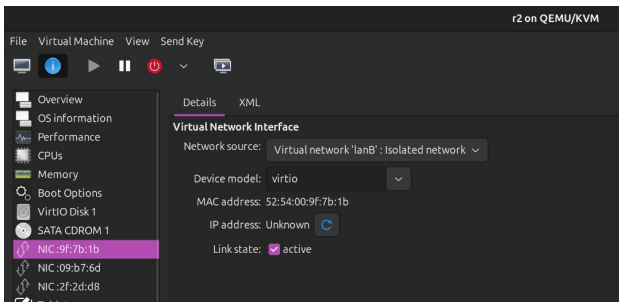
client: card on linkD



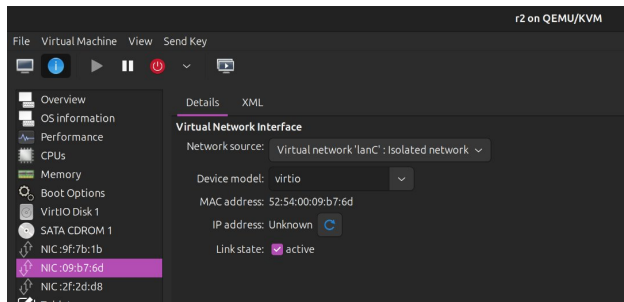
r1: card on lanA



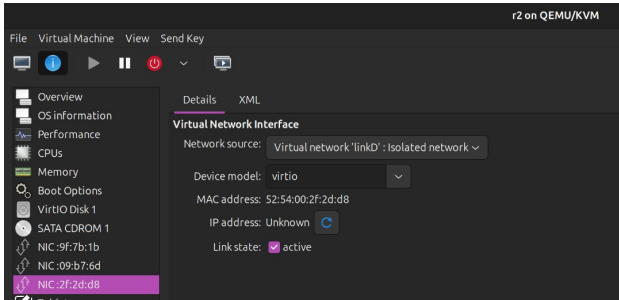
r1: card on lanB



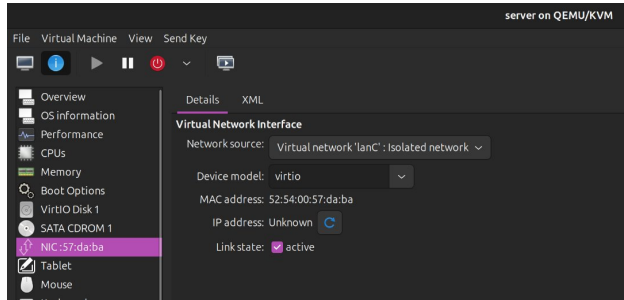
r2: card on lanB



r2: card on lanC



r2: card on linkD



server: card on lanC

## ip -br addr and ip route on each machine

```
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ ip -br addr
lo                UNKNOWN          127.0.0.1/8 ::1/128
enp1s0            UP                10.10.1.10/24
enp7s0            UP                10.10.4.2/30
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ ip route
10.10.1.0/24 dev enp1s0 proto kernel scope link src 10.10.1.10
10.10.2.0/24 via 10.10.1.1 dev enp1s0
10.10.3.0/24 via 10.10.1.1 dev enp1s0
10.10.4.0/30 dev enp7s0 proto kernel scope link src 10.10.4.2
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ this is client
```

client: ip -br addr and ip route

```
0 packets dropped by kernel
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ ip -br addr
lo                UNKNOWN          127.0.0.1/8 ::1/128
enp1s0            UP                10.10.1.1/24
enp7s0            UP                10.10.2.1/24
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ ip route
10.10.0.0/16 via 10.10.2.2 dev enp7s0
10.10.1.0/24 dev enp1s0 proto kernel scope link src 10.10.1.1
10.10.2.0/24 dev enp7s0 proto kernel scope link src 10.10.2.1
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ this is r1_
```

r1: ip -br addr and ip route

```

[edit] password for user:
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ ip -br addr
lo                UNKNOWN          127.0.0.1/8 ::1/128
enp1s0            UP                10.10.2.2/24
enp7s0            UP                10.10.3.1/24
enp8s0            UP                10.10.4.1/30
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ ip route
10.10.1.0/24 via 10.10.2.1 dev enp1s0
10.10.1.10 via 10.10.4.2 dev enp8s0
10.10.2.0/24 dev enp1s0 proto kernel scope link src 10.10.2.2
10.10.3.0/24 dev enp7s0 proto kernel scope link src 10.10.3.1
10.10.4.0/30 dev enp8s0 proto kernel scope link src 10.10.4.1
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ this is r2_

```

r2: ip -br addr and ip route

```

[edit] password for user:
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ ip -br addr
lo                UNKNOWN          127.0.0.1/8 ::1/128
enp1s0            UP                10.10.3.10/24
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ ip route
default via 10.10.3.1 dev enp1s0
10.10.3.0/24 dev enp1s0 proto kernel scope link src 10.10.3.10
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ this is server_

```

server: ip -br addr and ip route

### What these screenshots show

- **client** has two addresses, 10.10.1.10 on lanA and 10.10.4.2 on linkD. To reach lanB and lanC it sends packets to r1 (10.10.1.1) through lanA. It does not forward packets.
- **r1** is attached to lanA and lanB. It has only its two connected networks and one extra route, 10.10.0.0/16 via r2 (10.10.2.2). That single route covers every network r1 is not attached to.
- **r2** is attached to lanB, lanC and linkD. It has no default route. It has a special route for the single address 10.10.1.10 that goes straight to the client over linkD (via 10.10.4.2). The rest of lanA goes through r1.
- **server** is only on lanC and uses r2 (10.10.3.1) as its default gateway for everything else. It does not forward packets.
- r1 and r2 forward packets. The client and the server do not. Everything is set by hand with the ip and sysctl commands, with no DHCP and no routing programs.

## R2. Traceroute in both directions

```
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ traceroute -n 10.10.3.10
traceroute to 10.10.3.10 (10.10.3.10), 30 hops max, 60 byte packets
 1  10.10.1.1  1.548 ms  1.180 ms  1.141 ms
 2  10.10.4.1  1.624 ms  1.581 ms  1.690 ms
 3  10.10.3.10  2.363 ms  2.742 ms  2.719 ms
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ this is client_
```

client: traceroute -n 10.10.3.10

```
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ traceroute -n 10.10.1.10
traceroute to 10.10.1.10 (10.10.1.10), 30 hops max, 60 byte packets
 1  10.10.3.1  1.293 ms  1.229 ms  1.216 ms
 2  10.10.1.10  2.601 ms  2.587 ms  2.575 ms
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ this is server_
```

server: traceroute -n 10.10.1.10

### Client to server (three hops)

- **Hop 1, 10.10.1.1:** This is r1, the client's first router. The client sends the packet to r1 through lanA.
- **Hop 2, 10.10.4.1:** This is r2. r1 passes the packet on to r2 over lanB. Traceroute shows r2 with its linkD address, and we explain why below.
- **Hop 3, 10.10.3.10:** This is the server on lanC. It is the destination, so it answers the last probe.

### Server to client (two hops)

- **Hop 1, 10.10.3.1:** This is r2, the server's gateway. It is shown with its lanC address.
- **Hop 2, 10.10.1.10:** This is the client. r2 sends the packet straight to the client over linkD, so r1 is not used at all.

### Why the two directions are different

The routes are not the same in both directions on purpose. Going to the server, the client sends everything for lanB and lanC to r1, so the path is client, r1, r2, server. Coming back, the server sends everything to r2, and r2 has a direct link to the client (linkD). So r2 sends the reply directly to the client, and the path is only server, r2, client. That is why the return path has one hop fewer.

### Why r2 shows up as 10.10.4.1

Traceroute works by sending packets with a small TTL. When the TTL runs out at r2, r2 sends an ICMP "time exceeded" message back to the client (10.10.1.10). r2 has a route for that address that goes out of its linkD card, and Linux uses the address of the card it sends from as the source address. So the client sees 10.10.4.1, the linkD address, and not 10.10.2.2, the lanB address where the probe came in. In the other direction the probe from the server comes in on lanC and the reply goes out on lanC, so r2 shows up as 10.10.3.1.

### R3. Reverse-path filtering (rp\_filter)

All machines start with `rp_filter = 1` (strict). With strict filtering, the kernel drops a packet if the route back to its source address would use a different card than the one the packet arrived on. Our paths are not the same in both directions, so on two machines we turned it off (set it to 0). The setting is applied on all, default and each interface, in the `netcfg` scripts. On `r1` and the server we kept it strict, as the assignment asks.

| Machine | rp_filter  | Why                                                                                                                                                                          |
|---------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| client  | 0 (off)    | Packets from the server arrive on linkD (source 10.10.3.10), but the client's route to the server goes out on lanA. With strict filtering the kernel would drop every reply. |
| r2      | 0 (off)    | Packets from the client arrive on lanB (they come through r1), but r2's route back to the client goes out on linkD. Strict filtering would drop them.                        |
| r1      | 1 (strict) | r1 sees traffic in the same direction it would reply, on the same cards. Nothing is asymmetric at r1, so strict filtering does no harm and it must stay on.                  |
| server  | 1 (strict) | Packets from the client arrive on lanC, and the server's route back (default via r2) also uses lanC. So strict filtering works fine and it must stay on.                     |

## R4. Our client with a standard web server

On the server VM we ran Python's standard server (`python3 -m http.server 8000`) in the folder that holds `index.html` and `big.bin`. The file `big.bin` is 102,400 bytes, which is above the 100 KB limit. On the client VM we downloaded both files with our own client (`run-client`) and with `curl`, then compared the `md5sum` values.

```
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ cd ~/www
user@ubuntu-base:~/www$ ls -l big.bin
-rw-rw-r-- 1 user user 102400 Oct 10 05:49 big.bin
user@ubuntu-base:~/www$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.10.1.10 - - [10/Oct/2026 11:40:52] "GET /index.html HTTP/1.1" 200 -
10.10.1.10 - - [10/Oct/2026 11:41:04] "GET /big.bin HTTP/1.1" 200 -
10.10.1.10 - - [10/Oct/2026 11:41:16] "GET /index.html HTTP/1.1" 200 -
10.10.1.10 - - [10/Oct/2026 11:41:29] "GET /big.bin HTTP/1.1" 200 -
^C
Keyboard interrupt received, exiting.
user@ubuntu-base:~/www$ This is R4 Server_
```

server: the file size, and the standard server answering four requests (two from our client, two from curl)

```
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ sudo ./client/run-client http://10.10.3.10:8000/index.html -o c_index.html; echo "
exit=$?"
Connection established with 10.10.3.10:8000!
HTTP/1.0 200 OK
exit=0
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ sudo ./client/run-client http://10.10.3.10:8000/big.bin -o c_big.bin; echo "exit=$
?"
Connection established with 10.10.3.10:8000!
HTTP/1.0 200 OK
exit=0
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ curl -s http://10.10.3.10:8000/index.html -o k_index.html
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ curl -s http://10.10.3.10:8000/big.bin -o k_big.bin
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ md5sum c_index.html k_index.html c_big.bin k_big.bin
3ed2f77acf431b85499b5b7845d3faee c_index.html
3ed2f77acf431b85499b5b7845d3faee k_index.html
4aa61c8d3983456b76b604fd37daf69b c_big.bin
4aa61c8d3983456b76b604fd37daf69b k_big.bin
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ ls -l c_big.bin k_big.bin
-rw-r--r-- 1 root root 102400 Oct 10 11:41 c_big.bin
-rw-rw-r-- 1 user user 102400 Oct 10 11:41 k_big.bin
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ This is R4 Client
```

client: downloads with `run-client` (with exit codes), downloads with `curl`, and the `md5sum` of all four files

| File       | Our client   | curl         | Exit code | md5sum                                  |
|------------|--------------|--------------|-----------|-----------------------------------------|
| index.html | c_index.html | k_index.html | 0         | 3ed2f77acf431b85499b5b7845d3faee (same) |
| big.bin    | c_big.bin    | k_big.bin    | 0         | 4aa61c8d3983456b76b604fd37daf69b (same) |

Both downloads finished with exit code 0 and the status line *HTTP/1.0 200 OK* (that is what Python's server sends). The `md5sum` of each file from our client is exactly the same as the one from `curl`, and the sizes are the same (102,400 bytes for `big.bin`).

## R5. curl, wget and our client with our own server

Now the roles are reversed. On the server VM we ran our own server (sudo ./server/run-server -p 8080 -d ~/www). On the client VM we downloaded big.bin with curl, wget and our own client, one after the other (our server handles one connection at a time).

```
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ sudo ./server/run-server -p 8080 -d ~/www
Connection accepted from 10.10.1.10:52272!
Connection accepted from 10.10.1.10:47912!
Connection accepted from 10.10.1.10:62588!
^Cuser@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ this is R5 server
```

server: our server accepting the three connections

```
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ curl -s http://10.10.3.10:8080/big.bin -o curl_big.bin
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ wget -q http://10.10.3.10:8080/big.bin -O wget_big.bin
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ sudo ./client/run-client http://10.10.3.10:8080/big.bin -o my_big.bin; echo "exit=$?"
Connection established with 10.10.3.10:8080!
HTTP/1.1 200 OK
exit=0
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ md5sum curl_big.bin wget_big.bin my_big.bin
4aa61c8d3983456b76b604fd37daf69b  curl_big.bin
4aa61c8d3983456b76b604fd37daf69b  wget_big.bin
4aa61c8d3983456b76b604fd37daf69b  my_big.bin
user@ubuntu-base:~/asgn3/COL334-Raw-TCP-HTTP$ this is R5 client _
```

client: curl, wget and run-client downloads, exit code, and md5sum of the three files

| Downloaded by | File         | md5sum                           |
|---------------|--------------|----------------------------------|
| curl          | curl_big.bin | 4aa61c8d3983456b76b604fd37daf69b |
| wget          | wget_big.bin | 4aa61c8d3983456b76b604fd37daf69b |
| our client    | my_big.bin   | 4aa61c8d3983456b76b604fd37daf69b |

Our server answered all three clients. Our client printed *HTTP/1.1 200 OK* and exited with code 0. All three md5sum values are the same, and they are also the same as the md5sum of big.bin in R4, which is the same 102,400-byte file in ~/www on the server.

## R6. Packet capture and how our TCP works

We captured one full connection of our client with tcpdump on the client VM. The client downloaded a small page from our server. The 'Out' lines leave through enp1s0 (lanA) and the 'In' lines arrive through enp7s0 (linkD), which shows the different paths from R2.

```
user@ubuntu-base:~/asn3/COL334-Raw-TCP-HTTP$ sudo tcpdump -nn -r /tmp/r6_capture.pcap
reading from file /tmp/r6_capture.pcap, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144
Warning: interface names might be incorrect
11:56:17.790147 enp1s0 Out IP 10.10.1.10.64496 > 10.10.3.10.8000: Flags [S], seq 49703, win 65535, options [mss 1460], length 0
11:56:17.791946 enp7s0 In IP 10.10.3.10.8000 > 10.10.1.10.64496: Flags [S.], seq 30571, ack 49704, win 65535, options [mss 1460], length 0
11:56:17.792174 enp1s0 Out IP 10.10.1.10.64496 > 10.10.3.10.8000: Flags [.], ack 1, win 65535, length 0
11:56:17.796917 enp1s0 Out IP 10.10.1.10.64496 > 10.10.3.10.8000: Flags [P.], seq 1:71, ack 1, win 65535, length 70
11:56:17.798258 enp7s0 In IP 10.10.3.10.8000 > 10.10.1.10.64496: Flags [.], ack 71, win 65535, length 0
11:56:17.798428 enp7s0 In IP 10.10.3.10.8000 > 10.10.1.10.64496: Flags [P.], seq 1:125, ack 71, win 65535, length 124
11:56:17.798649 enp1s0 Out IP 10.10.1.10.64496 > 10.10.3.10.8000: Flags [.], ack 125, win 65535, length 0
11:56:17.799584 enp7s0 In IP 10.10.3.10.8000 > 10.10.1.10.64496: Flags [F.], seq 125, ack 71, win 65535, length 0
11:56:17.799679 enp1s0 Out IP 10.10.1.10.64496 > 10.10.3.10.8000: Flags [.], ack 126, win 65535, length 0
11:56:17.799758 enp1s0 Out IP 10.10.1.10.64496 > 10.10.3.10.8000: Flags [F.], seq 71, ack 126, win 65535, length 0
11:56:17.800310 enp7s0 In IP 10.10.3.10.8000 > 10.10.1.10.64496: Flags [.], ack 72, win 65535, length 0
11:56:19.802856 enp1s0 Out IP 10.10.1.10.64496 > 10.10.3.10.8000: Flags [.], ack 126, win 65535, length 0
user@ubuntu-base:~/asn3/COL334-Raw-TCP-HTTP$
```

client: tcpdump -nn -r /tmp/r6\_capture.pcap (one complete connection)

### What happens in the capture

- **Handshake:** The client sends a SYN (seq 49703) with MSS 1460. The server answers with a SYN-ACK (seq 30571, ack 49704), also with MSS 1460. The client sends an ACK. Both sides advertise window 65535.
- **Data:** The client sends the HTTP request (70 bytes, flags P.). The server acknowledges it (ack 71) and sends its response (124 bytes). The client acknowledges it (ack 125). One ACK is sent for each data segment.
- **Closing:** The server sends its FIN (seq 125) and the client ACKs it (ack 126). The client then sends its own FIN (seq 71) and the server ACKs it (ack 72). The last ACK in the list comes about 2 seconds later, because our close() waits up to 2 seconds for the other side's FIN before it finishes.
- This transfer was small and clean, so no packet was lost and there is no retransmission in this capture. The next parts describe what our code does when something goes wrong.

### Retransmission

We check for lost packets with a timer of 1 second. When nothing useful arrives for 1 second while we are sending, we assume the data was lost. We cut the window (sssthresh becomes half of cwnd, but never less than 2 MSS, and cwnd goes back to 1 MSS) and send the data again from the oldest byte that is not acknowledged. The timer stays at 1 second (we do not make it longer after each timeout). If nothing makes progress for 10 timeouts in a row (about 10 seconds), we give up. We also use **fast retransmit**: after 3 duplicate ACKs, we send the missing segment again at once without waiting for the timer. The handshake sends the SYN again every 1 second and gives up after 6 tries (6 seconds), and the FIN is also sent again until it is acknowledged, but we stop waiting after about 2 seconds. If a RST arrives, we drop the connection immediately.

### Out-of-order buffering

The receiver keeps the next byte it expects (the ack number). If a segment arrives exactly where we expect, we keep its data, move the ack number forward, and ACK it. If a segment arrives ahead of the expected byte (there is a gap), we do not throw it away. We store it in a dictionary, with its sequence number as the key, and we send an ACK that repeats the same ack number (a duplicate ACK). When the missing segment finally arrives, a loop takes out all the stored segments that now follow one another, adds them to the data and moves the ack number past them. Old duplicate

segments are not added again, so the data goes to HTTP in order and only once. We also never let the ack number go backwards.

### Congestion control (TCP Reno style)

- We start with  $cwnd = 1 \text{ MSS}$  (1460 bytes) and  $ssthresh = 65535$  bytes.
- **Slow start:** while  $cwnd$  is below  $ssthresh$ ,  $cwnd$  grows by 1 MSS for every new ACK, so it roughly doubles each round trip.
- **Congestion avoidance:** once  $cwnd$  reaches  $ssthresh$ , it grows by about  $MSS \times MSS / cwnd$  per ACK, which is about 1 MSS per round trip.
- **Loss by timeout:**  $ssthresh = \max(cwnd / 2, 2 \text{ MSS})$  and  $cwnd = 1 \text{ MSS}$ , so we start slow start again.
- **Loss by 3 duplicate ACKs:**  $ssthresh = \max(cwnd / 2, 2 \text{ MSS})$ ,  $cwnd = ssthresh + 3 \text{ MSS}$ , and the missing segment is sent again. This is a simple form of fast recovery. We do not keep increasing the window for further duplicate ACKs.
- We never have more bytes in flight than the smaller of  $cwnd$  and the window the other side advertises. Our own receive window is fixed at 65535.

### Extra notes on the VM setup

- Because the kernel knows nothing about our raw-socket connections, it would send RST packets for them. On the client and server VMs we block these with `iptables -A OUTPUT -p tcp --tcp-flags RST RST -j DROP`.
- The virtual network cards use checksum offloading, so the checksum bytes that a raw socket sees can be wrong. We turned it off on the client and server with `ethtool -K <interface> rx off tx off`. Our code still checks every checksum and drops bad packets.